

ODT Library

Linear Layer and RigL Implementation

A Complete Line-by-Line Guide

Files: Linear.h Linear.c LinearApi.h LinearApi.c

Topics: Forward pass, Backward pass, Weight/Bias/PropLoss gradients, Quantization dispatch, Kaiming init, Borrowing vs Owning factories, Transpose trick, SYM_INT32 fixed-point math, VLA stack allocation, Arena allocator, RigL hook integration

Chapter 1 -- Architecture Overview

A linear layer implements $y = x * W^T + b$. Files: Linear.h (declarations), Linear.c (math), LinearApi.h (factory decls), LinearApi.c (factory impls).

CONCEPT: Three-Tensor Model

Every linear layer owns:

1. Weights W: shape [outFeatures, inFeatures]. param tensor + grad tensor.
2. Bias b: shape [outFeatures]. param tensor + grad tensor.
3. Four quantization descriptors: forwardQ, weightGradQ, biasGradQ, propLossQ.

Forward: `input [batch,inF] * W^T [inF,outF] + b [outF] = output [batch,outF]`

Backward: `weightGrad += loss^T * input (outer product)`

`biasGrad += sum(loss, axis=0) (reduce batch)`

`propLoss = loss * W (gradient to previous layer)`

Chapter 2 -- linearConfig_t Struct

Central control block for one linear layer. Holds two parameter bundles and four quantization pointers.

typedef struct linearConfig {	
parameter_t *weights;	<-- weights param+grad bundle
parameter_t *bias;	<-- bias param+grad bundle
quantization_t *forwardQ;	<-- math type for forward matmul
quantization_t *weightGradQ;	<-- math type for weight grad accumulation
quantization_t *biasGradQ;	<-- math type for bias grad accumulation
quantization_t *propLossQ;	<-- math type for propagated loss
bool ownsQuantizations;	<-- true: freeLinearLayer frees quant structs
} linearConfig_t;	

CONCEPT: parameter_t -- Weights and Gradients Together

parameter_t bundles param tensor with its grad tensor. Both same shape.

```
struct parameter { tensor_t *param; tensor_t *grad; };
```

getParamFromParameter(p) returns p->param

getGradFromParameter(p) returns p->grad

CONCEPT: ownsQuantizations Flag

Borrowing (false): stores raw pointers. Caller owns quant structs. freeLinearLayer skips them.

Owning (true): deep-copies all four quantization_t. Caller can free originals immediately.

freeLinearLayer reads this flag to decide whether to free quant structs.

Chapter 3 -- linearInitConfig (Legacy)

Simplest function -- copies six pointers into the struct. No allocation, no validation.

void linearInitConfig(linearConfig_t *cfg,	
parameter_t *weights,	
parameter_t *bias, quantization_t *forwardQ,	
quantization_t *weightGradQ, quantization_t	
*biasGradQ,	
quantization_t *propLossQ) {	
cfg->weights = weights;	<-- store pointer, no copy
cfg->bias = bias;	<-- store pointer, no copy
cfg->forwardQ = forwardQ;	
cfg->weightGradQ = weightGradQ;	
cfg->biasGradQ = biasGradQ;	
cfg->propLossQ = propLossQ;	
}	

Chapter 4 -- Forward Pass

Computes $\text{output} = \text{input} * W^T + b$. Weights stored as $[\text{outF}, \text{inF}]$. For matmul we need $[\text{inF}, \text{outF}]$, so we transpose W (zero-copy metadata swap), multiply, then restore.

<code>void linearForwardFloat(tensor_t *w, tensor_t *b,</code>	
<code>tensor_t *input, tensor_t *output) {</code>	
<code>transposeTensor(w, 0, 1);</code>	<code><-- flip dims: [outF,inF]->[inF,outF]</code> (metadata only)
<code>matmulFloat32TensorsWithBias(input, w, output, b);</code>	<code><-- output = input*W^T + b</code>
<code>transposeTensor(w, 0, 1);</code>	<code><-- restore [outF,inF]</code>
<code>}</code>	

CONCEPT: Zero-Copy Transpose Trick

`transposeTensor(w,0,1)` swaps $w \rightarrow \text{shape} \rightarrow \text{orderOfDimensions}[0]$ and $[1]$.

NO data moves. Only 2 integers in shape metadata are swapped.

Before: $\text{orderOfDimensions}=[0,1]$, $\text{dims}=[\text{outF}, \text{inF}] \rightarrow \text{row } i = \text{neuron } i \text{ weights}$

After: $\text{orderOfDimensions}=[1,0]$, $\text{dims}=[\text{outF}, \text{inF}] \rightarrow \text{calcElementIndexByIndices}$ permutes indices

logically reads as $[\text{inF}, \text{outF}]$ without any memcopy

Two metadata swaps, zero bytes moved. Critical on 320 KB MCU.

$W [2,3]$ physical: After transpose (same bytes, different index mapping):

$[\text{w00 } \text{w01 } \text{w02}] \text{ row0:}[\text{w00 } \text{w10}] \text{ row1:}[\text{w01 } \text{w11}] \text{ row2:}[\text{w02 } \text{w12}]$

$[\text{w10 } \text{w11 } \text{w12}]$

$\text{input } [\text{batch}, 3] * W^T [3, 2] = \text{output } [\text{batch}, 2]$

<code>void linearForwardSymInt32(tensor_t *w, tensor_t</code>	
<code>*b,</code>	
<code>tensor_t *input, tensor_t *output) {</code>	
<code>transposeTensor(w, 0, 1);</code>	<code><-- zero-copy transpose</code>
<code>matmulSymInt32TensorsWithBias(input, w, output,</code>	<code><-- fixed-point matmul</code>
<code>b);</code>	
<code>transposeTensor(w, 0, 1);</code>	<code><-- restore</code>
<code>}</code>	

CONCEPT: SYM_INT32 Fixed-Point Math

$\text{real} = \text{mantissa} * \text{scale}$. mantissa : `int32_t`. scale : float in `symInt32QConfig_t`.

$\text{qBits}=8$ for weight storage. $\text{qMaxBits}=16$ for training (avoids overflow).

`matmulSymInt32` auto-tracks output scale: $\text{out_scale} = A_scale * B_scale$.

This is the FQT (Fixed-point Quantization Training) core mechanism.

void linearForward(layer_t *linearLayer,	
tensor_t *input, tensor_t *output) {	
linearConfig_t *cfg = linearLayer->config->linear;	<-- unpack config
tensor_t *w = getParamFromParameter(cfg->weights);	<-- get param tensor
tensor_t *b = getParamFromParameter(cfg->bias);	<-- get bias tensor
switch (cfg->forwardQ->type) {	<-- dispatch on qtype
case FLOAT32:	
linearForwardFloat(w, b, input, output); break;	<-- float path
case SYM_INT32:	
linearForwardSymInt32(w, b, input, output); break;	<-- fixed-point path
default: PRINT_ERROR("Unknown QType!"); exit(1);	<-- fail fast
}	
}	

CONCEPT: Why exit(1)?

No OS on MCU. Unknown qtype = impossible state.

exit(1) halts immediately before undefined behavior corrupts memory.

PRINT_ERROR sends message over UART for serial monitor diagnosis.

Chapter 5 -- Weight Gradient Computation

$dW = loss^T * input$. Accumulated (never overwritten) into weightGrads.

void linearCalcWeightGradsFloat32(tensor_t *forwardInput,	
tensor_t *loss, tensor_t *weightGrads) {	
size_t numberOfWeights =	<-- count
calcNumberOfElementsByTensor(weightGrads);	
tensor_t intermediateWGrad;	<-- temp tensor on stack
float intermediateWGradData[numberOfWeights];	<-- VLA: C99 runtime-sized stack buffer
quantization_t intermediateWGradQ;	
initFloat32Quantization(&intermediateWGradQ);	
setTensorValues(&intermediateWGrad, (uint8_t *)intermediateWGradData,	
weightGrads->shape, &intermediateWGradQ,	<-- wire struct
weightGrads->sparsity);	
transposeTensor(loss, 0, 1);	<-- loss:[batch,outF]->[outF,batch]
matmulFloat32Tensors(loss, forwardInput,	<-- $dW = loss^T * input$
&intermediateWGrad);	
transposeTensor(loss, 0, 1);	<-- restore loss layout
addFloat32TensorsInplace(weightGrads,	<-- weightGrads += dW
&intermediateWGrad);	
}	

CONCEPT: Why an Intermediate Buffer?

weightGrads may contain gradients from previous mini-batches (gradient accumulation).

Pattern: compute dW into fresh temp -> ADD into weightGrads.

This supports multi-batch gradient accumulation without extra logic.

CONCEPT: VLA Risk on MCU

float intermediateWGradData[numberOfWeights] is a C99 VLA.

Allocated on STACK at runtime. No compile-time size check.

RISK: $1024 \times 256 = 262144$ floats = 1 MB -> stack overflow on 320 KB MCU.

Stack grows down from _estack. No MMU protection on Cortex-M7.

Stack/heap collision is silent crash. Always budget 4 bytes x numberOfWeights.

void linearCalcWeightGradsFloatWithConversion(	
linearConfig_t *cfg, tensor_t *forwardInput,	
tensor_t *loss) {	

tensor_t *paramWG = getGradFromParameter(cfg->weights);	<-- permanent grad tensor
tensor_t *wG = paramWG;	<-- working ptr, may be redirected to local converted copy
tensor_t *fwdIn = forwardInput; tensor_t *l = loss;	<-- working ptrs
float forwardInputFloatData[calcNumberOfElementsBy Tensor(forwardInput)];	<-- VLA
tensor_t forwardInputFloat; quantization_t forwardInputFloatQ;	<-- local targets
initFloat32Quantization(&forwardInputFloatQ);	
if (fwdIn->quantization->type != FLOAT32) {	<-- only convert if needed
setTensorValuesForConversion((uint8_t)forwardInputFloatData,	<-- wire VLA into struct
&forwardInputFloatQ, fwdIn, &forwardInputFloat);	
convertTensor(forwardInput, &forwardInputFloat);	<-- fill with converted values
fwdIn = &forwardInputFloat;	<-- redirect working ptr
}	
/* same pattern for loss and weightGrad */	
linearCalcWeightGradsFloat32(fwdIn, l, wG);	<-- all FLOAT32 now
convertTensor(wG, paramWG);	<-- convert result back to original type
}	

CONCEPT: Pointer Redirect Pattern

tensor_t *fwdIn = forwardInput; -- starts at original

if (type != FLOAT32) { fwdIn = &forwardInputFloat; } -- redirect to local

The math function only sees fwdIn -- never knows if conversion happened.

Reassigning local working pointer does NOT affect caller's pointer.

Clean separation: conversion logic in wrapper, math logic in pure function.

CONCEPT: setTensorValuesForConversion -- Stack Buffer to tensor_t

setTensorValuesForConversion(data, q, src, dst):

dst uses data as its storage (the VLA on stack)

dst uses q as its quantization descriptor

dst inherits shape and sparsity from src

Then convertTensor(src, dst) fills that buffer.

No malloc. Temporary tensor vanishes when function returns.

Chapter 6 -- Bias Gradient Computation

$dB[f] = \sum_n \text{loss}[n,f]$. Sum loss over batch dimension for each feature f .

<code>void linearCalcBiasGradsFloat32(tensor_t *loss, tensor_t *biasGrad) {</code>	
<code>size_t numFeatures = calcNumberOfElementsByTensor(biasGrad);</code>	<code><-- outF</code>
<code>size_t numLoss = calcNumberOfElementsByTensor(loss);</code>	<code><-- batch * outF</code>
<code>size_t batch = (numFeatures==0)?0:numLoss/numFeatures;</code>	<code><-- batch size</code>
<code>float *bg = (float *)biasGrad->data;</code>	<code><-- cast to float*</code>
<code>float *l = (float *)loss->data;</code>	<code><-- cast to float*</code>
<code>for (size_t f=0; f<numFeatures; f++) {</code>	<code><-- for each output feature</code>
<code>float sum = 0.0f;</code>	<code><-- accumulator</code>
<code>for (size_t n=0; n<batch; n++) {</code>	<code><-- for each batch element</code>
<code>sum += l[n * numFeatures + f];</code>	<code><-- loss[n,f] in row-major: flat = n*cols+f</code>
<code>}</code>	
<code>bg[f] += sum;</code>	<code><-- ACCUMULATE not overwrite</code>
<code>}</code>	
<code>}</code>	

Example: batch=2, outF=3

loss [2,3]: row0=[0.1,0.2,0.3] row1=[0.4,0.5,0.6]

f=0: sum=0.1+0.4=0.5 -> biasGrad[0]+=0.5

f=1: sum=0.2+0.5=0.7 -> biasGrad[1]+=0.7

f=2: sum=0.3+0.6=0.9 -> biasGrad[2]+=0.9

Row-major indexing: $\text{loss}[n,f] = l[n * \text{numFeatures} + f]$

<code>void linearCalcBiasGradsSymInt32(tensor_t *biasGrads, tensor_t *loss) {</code>	
<code>/* numFeatures, batch, int32 pointers same as float version */</code>	
<code>float lossScale=((symInt32QConfig_t*)loss->quantization->qConfig)->scale;</code>	
<code>float bgScale=((symInt32QConfig_t*)biasGrads->quantization->qConfig)->scale;</code>	
<code>for (size_t f=0; f<numFeatures; f++) {</code>	
<code>int32_t sum=0;</code>	<code><-- int32 (NOT int64) accumulator</code>
<code>for (size_t n=0; n<batch; n++) {</code>	
<code>sum += l[n*numFeatures+f];</code>	<code><-- accumulate mantissas</code>
<code>}</code>	
<code>bg[f] += (int32_t)roundf((float)sum * lossScale / bgScale);</code>	<code><-- rescale+accumulate</code>
<code>}</code>	

```
}
```

CONCEPT: Why int32 Not int64 for SYM_INT32 Accumulation?

qMaxBits=16 -> mantissas fit in $[-32767, +32767]$ (int16 range).

Worst case: $65536 \text{ batch} * 32767 = \sim 2.1\text{e}9 < \text{INT32_MAX} = 2.147\text{e}9$.

int32 safe for any realistic batch size.

int64 would be 2x memory and slower (software-emulated on Cortex-M7).

CONCEPT: Scale Correction in SYM_INT32 Bias Gradient

Sum is in loss-scale units. Must convert to biasGrad-scale units.

$\text{mantissa_bg} = \text{round}(\text{sum_mantissa} * \text{lossScale} / \text{bgScale})$

Example: $\text{sum}=100, \text{lossScale}=0.01, \text{bgScale}=0.05$

$\text{real sum} = 100 * 0.01 = 1.0 \rightarrow \text{mantissa_bg} = \text{round}(1.0 / 0.05) = 20$

FQT core pattern: all arithmetic stays integer, scale correction at end.

Chapter 7 -- Propagated Loss

propLoss = loss * W. Shape [batch,inF]. Flows upstream to previous layer.

```
void linearCalcPropLossFloat32(tensor_t *loss,
    tensor_t *weights, tensor_t *propLoss) {
    matmulFloat32Tensors(loss, weights, propLoss);
}
```

<-- propLoss = loss * W

```
loss: [batch, outF]
weights: [outF, inF] (NOT transposed, used as-is)
propLoss: [batch, inF] <- gradient flowing to previous layer
matmul: [batch,outF] * [outF,inF] = [batch,inF] OK
```

```
void linearCalcPropLossFloatWithConversion(...) {
    if (propLoss->quantization->type != FLOAT32) {
        PRINT_ERROR("propLossQ is FLOAT32 but tensor is
            not (#187)");
        exit(1);
    }
    /* convert weights and loss to FLOAT32 if needed
    */
    linearCalcPropLossFloat32(l, w, pL);
}
```

<-- CRITICAL safety check

<-- fail fast before NULL dereference
segfault

WARNING: Issue #187 -- propLoss Type Guard

matmulFloat32Tensors writes raw float bits into propLoss->data. If propLoss is SYM_INT32 its qConfig->scale may be NULL, causing NULL dereference when matmul tries to update output scale. Guard catches this at API boundary with clear UART error message.

Chapter 8 -- Full Backward Pass

8.1 -- backwardFloat

void backwardFloat(linearConfig_t *cfg, tensor_t *forwardInput,	
tensor_t *loss, tensor_t *propLossTensor) {	
size_t numberOfWeights =	
calcNumberOfElementsByShape(cfg->weights->param->shape);	<-- count weights
tensor_t *weightGrad =	<-- permanent grad
getGradFromParameter(cfg->weights);	
tensor_t *biasGrad =	<-- permanent bias grad
getGradFromParameter(cfg->bias);	
tensor_t intermediateWGrad;	<-- temp buffer
uint8_t intermediateWGradData[numberOfWeights * sizeof(float)];	<-- VLA as uint8_t
setTensorValues(&intermediateWGrad, intermediateWGradData,	<-- wire struct
weightGrad->shape, weightGrad->quantization, weightGrad->sparsity);	
linearCalcWeightGradsFloat32(forwardInput, loss, &intermediateWGrad);	<-- compute dW
addFloat32TensorsInplace(weightGrad, &intermediateWGrad);	<-- weightGrad += dW
linearCalcBiasGradsFloat32(loss, biasGrad);	<-- biasGrad += sum(loss)
tensor_t *w = getParamFromParameter(cfg->weights);	
linearCalcPropLossFloat32(loss, w, propLossTensor);	<-- propLoss = loss*W
}	

Note: uint8_t VLA (not float[]) because setTensorValues expects uint8_t*. The tensor reinterprets bytes as float via its quantization type.

8.2 -- backwardSymInt32

void backwardSymInt32(linearConfig_t *cfg, tensor_t *forwardInput,	
tensor_t *loss, tensor_t *propLoss) {	
tensor_t *weights =	
getParamFromParameter(cfg->weights);	
tensor_t *weightGrads =	
getGradFromParameter(cfg->weights);	
tensor_t *biasGrads =	
getGradFromParameter(cfg->bias);	
symInt32QConfig_t *wgQC =	<-- get rounding mode
weightGrads->quantization->qConfig;	

symInt32QConfig_t intermediateQC;	
initSymInt32QConfig(wgQC->roundingMode, &intermediateQC);	<-- copy rounding mode
quantization_t intermediateQ;	
initSymInt32Quantization(&intermediateQC, &intermediateQ);	
int32_t intermediateData[numberOfWeights];	<-- VLA int32 buffer
setTensorValues(&intermediate, (uint8_t*)intermediateData,	
weightGrads->shape, &intermediateQ, NULL);	<-- sparsity=NULL for temp
linearCalcWeightGradsSymInt32(loss, forwardInput, &intermediate);	
addSymInt32TensorsInplace(weightGrads, &intermediate);	
linearCalcBiasGradsSymInt32(biasGrads, loss);	
linearCalcPropLossSymInt32(weights, loss, propLoss);	
}	

CONCEPT: Why Copy roundingMode?

Intermediate buffer must use same rounding mode as permanent grad tensor.

Different modes -> different quantization bias in accumulated gradients.

Copying wgQC->roundingMode makes intermediate a perfect temporary clone.

8.3 -- linearBackward (dispatch)

void linearBackward(layer_t *linearLayer, tensor_t *forwardInput,	
tensor_t *loss, tensor_t *propLoss) {	
linearConfig_t *cfg = linearLayer->config->linear;	
switch (cfg->weightGradQ->type) {	<-- dispatch weight grad path
case FLOAT32:	
linearCalcWeightGradsFloatWithConversion(cfg, forwardInput, loss); break;	
case SYM_INT32:	
linearCalcWeightGradsSymInt32WithConversion(cfg, loss, forwardInput); break;	
}	
switch (cfg->biasGradQ->type) { /* FLOAT32 or SYM_INT32 */ }	
switch (cfg->propLossQ->type) { /* FLOAT32 or SYM_INT32 */ }	
}	

Three independent switch statements. Each sub-operation can use a different quantization type -- e.g. weight grads in SYM_INT32 but propLoss in FLOAT32.

Chapter 9 -- linearCalcOutputShape

<code>void linearCalcOutputShape(layer_t *linearLayer,</code>	
<code>shape_t *inputShape, shape_t *outputShape) {</code>	
<code>if (inputShape->numberOfDimensions != 2) {</code>	<code><-- linear needs [batch,inF]</code>
<code>PRINT_ERROR("Linear expects 2D input, got %luD",</code> <code>...);</code>	
<code>}</code>	
<code>size_t batchSize = inputShape->dimensions[0];</code>	<code><-- first dim = batch</code>
<code>shape_t *wShape = cfg->weights->param->shape;</code>	<code><-- weight shape [outF,inF]</code>
<code>size_t outFeatures = wShape->dimensions[0];</code>	<code><-- outF from dim 0</code>
<code>outputShape->dimensions[0] = batchSize;</code>	<code><-- output: [batch, outF]</code>
<code>outputShape->dimensions[1] = outFeatures;</code>	
<code>outputShape->numberOfDimensions = 2;</code>	
<code>setOrderOfDimsForNewTensor(2,</code> <code>outputShape->orderOfDimensions);</code>	<code><-- [0,1] = normal</code>
<code>}</code>	

Example: inF=512, outF=64, batch=8

inputShape: [8, 512]

weightShape: [64, 512] (64 neurons, each with 512 weights)

outputShape: [8, 64] (8 batch elements, 64 outputs each)

Chapter 10 -- LinearApi Factory Functions

Two generations: legacy API (manual parameter setup) and new factory API (linearLayerInit).

10.1 -- linearLayerInitLegacy

layer_t *linearLayerInitLegacy(parameter_t *weights, parameter_t *bias,	
quantization_t *forwardQ, quantization_t *weightGradsQ,	
quantization_t *biasGradsQ, quantization_t *propLossQ) {	
layer_t *linearLayer = reserveMemory(sizeof(layer_t));	<-- arena alloc
linearLayer->type = LINEAR;	<-- tag as linear
layerConfig_t *layerCfg = reserveMemory(sizeof(layerConfig_t));	
linearConfig_t *linCfg = reserveMemory(sizeof(linearConfig_t));	
layerCfg->linear = linCfg;	<-- wire config
linCfg->weights = weights;	<-- store caller pointer
linCfg->bias = bias;	
linCfg->forwardQ = forwardQ;	
linCfg->weightGradQ = weightGradsQ;	
linCfg->biasGradQ = biasGradsQ;	
linCfg->propLossQ = propLossQ;	
linCfg->ownsQuantizations = false;	<-- caller owns quant structs
linearLayer->config = layerCfg;	
return linearLayer;	
}	

CONCEPT: reserveMemory -- The ODT Arena Allocator

NOT malloc. Bump-pointer arena allocator:

```
static uint8_t arena[ARENA_SIZE]; static size_t bump=0;
```

```
void* reserveMemory(n) { void *p=&arena[bump]; bump+=align(n); return p; }
```

O(1) allocation. No fragmentation. Zero-initialized (calloc semantics).

Linker verifies ARENA_SIZE fits in SRAM at compile time.

freeReservedMemory(ptr) only works for most recent allocation.

10.2 -- linearLayerInitNonTrainableLegacy (inference only)

layer_t	
*linearLayerInitNonTrainableLegacy(tensor_t	
*weights, tensor_t *bias,	
quantization_t *forwardQ) {	
/* ... layer/config allocation ... */	
linCfg->weights = parameterInit(weights, NULL);	<-- grad=NULL: not trainable
linCfg->bias = parameterInit(bias, NULL);	<-- grad=NULL: not trainable
linCfg->forwardQ = forwardQ;	<-- only forward path configured
}	

parameterInit(param, NULL) creates parameter_t with grad=NULL. getGradFromParameter returns NULL. Backward must not be called.

Chapter 11 -- New Factory API (linearLayerInit)

New factory API uses linearInit_t (shape params) + layerQuant_t (quantization profile). Factory handles all allocation and Kaiming weight init internally.

typedef struct linearInit {	
size_t inFeatures;	<-- REQUIRED: input dimension
size_t outFeatures;	<-- REQUIRED: number of output neurons
bias_t bias;	<-- OPTIONAL: BIAS_DEFAULT/TRUE/FALSE
} linearInit_t;	

CONCEPT: bias_t and resolveLinearBias

bias_t enum: BIAS_DEFAULT=0, BIAS_TRUE, BIAS_FALSE

resolveLinearBias(BIAS_DEFAULT)->true (PyTorch parity)

resolveLinearBias(BIAS_TRUE)->true resolveLinearBias(BIAS_FALSE)->>false

Enum (not bool) allows DEFAULT state inheriting framework convention.

11.1 -- buildOwnedShape (internal helper)

static shape_t *buildOwnedShape(const size_t *srcDims, size_t numberOfDims) {	
size_t *dims = reserveMemory(numberOfDims * sizeof(size_t));	<-- alloc dims array
for (size_t i=0; i<numberOfDims; i++) { dims[i]=srcDims[i]; }	<-- copy
size_t *order = reserveMemory(numberOfDims * sizeof(size_t));	<-- alloc order array
setOrderOfDimsForNewTensor(numberOfDims, order);	<-- order=[0,1,...,N-1] normal
shape_t *shape = reserveMemory(sizeof(shape_t));	
setShape(shape, dims, numberOfDims, order);	<-- wire everything
return shape;	
}	

CONCEPT: setOrderOfDimsForNewTensor

Sets orderOfDimensions to identity [0,1,...,N-1]. New tensor always normal (not transposed).

transposeTensor() swaps entries later to change logical access order.

11.2 -- allocateLinearWeights (Kaiming init)

static parameter_t *allocateLinearWeights(size_t inF, size_t outF,	
quantization_t *storageQ, quantization_t *gradQ) {	
shape_t *shape = buildOwnedShape((size_t[]){outF,inF}, 2);	<-- [outF,inF]
tensor_t *paramTensor = initTensor(shape, getQLike(storageQ), NULL);	<-- alloc tensor
distribution_t dist = {	
.type = KAIMING_UNIFORM,	<-- He et al. 2015
.params.kaiming={.gain=1.4142135623730951f, .fanMode=inF},	<-- sqrt(2), fan_in
};	
initDistribution(paramTensor, &dist);	<-- fill with Kaiming uniform values
tensor_t *gradTensor = gradInit(paramTensor, gradQ, NULL);	<-- alloc grad tensor
return parameterInit(paramTensor, gradTensor);	<-- bundle param+grad
}	

CONCEPT: Kaiming Uniform Initialization (He et al. 2015)

$W \sim \text{Uniform}[-\text{bound}, +\text{bound}]$ where $\text{bound} = \text{gain} * \sqrt{1/\text{fanMode}}$

$\text{fanMode} = \text{inFeatures}$ (fan_in mode). $\text{gain} = \sqrt{2}$ for ReLU.

Why Kaiming? Prevents vanishing/exploding gradients in deep networks.

Scales init variance to layer size, keeping signal magnitude stable.

PyTorch uses $a=\sqrt{5}$ (different gain).

ODT uses $\sqrt{2}$. Issue C will add parametrization for bit-identical parity.

CONCEPT: getQLike -- Cloning a Quantization Descriptor

`getQLike(q)` allocates new `quantization_t` with same type and `qConfig` as `q`.

Tensor takes ownership of its own quantization copy.

Caller can discard original `q` after `initTensor` returns.

11.3 -- allocateLinearBias

static parameter_t *allocateLinearBias(size_t outF,	
quantization_t *storageQ, quantization_t *gradQ) {	
shape_t *shape = buildOwnedShape((size_t[]){outF}, 1);	<-- [outF] 1D tensor

tensor_t *paramTensor = initTensor(shape, getQLike(storageQ), NULL);	
/* No fill needed: reserveMemory = calloc, data already 0 */	
tensor_t *gradTensor = gradInit(paramTensor, gradQ, NULL);	
return parameterInit(paramTensor, gradTensor);	
}	

reserveMemory is zero-initialized (calloc semantics). Bias starts at 0 automatically.

11.4 -- linearLayerInit (Borrowing)

layer_t *linearLayerInit(linearInit_t *init, layerQuant_t *lq) {	
validateLinearInit(init);	<-- check inF>0, outF>0
bool hasBias = resolveLinearBias(init->bias);	
validateLayerQuantForLinear(lq, hasBias);	<-- check pointers non-NULL
/* alloc layer, layerConfig, linearConfig */	
cfg->weights = allocateLinearWeights(init->inFeatures, init->outFeatures,	
lq->weightStorage, lq->backwardMath);	<-- Kaiming init + grad alloc
cfg->bias = hasBias ? allocateLinearBias(...) : NULL;	<-- NULL if BIAS_FALSE
cfg->forwardQ = lq->forwardMath;	<-- BORROW: store raw pointer
cfg->weightGradQ = lq->backwardMath;	<-- BORROW: store raw pointer
cfg->biasGradQ = lq->backwardMath;	<-- BORROW: store raw pointer
cfg->propLossQ = lq->backwardMath;	<-- BORROW: store raw pointer
cfg->ownsQuantizations = false;	
return layer;	
}	

11.5 -- linearLayerInitOwning (deep-copy)

layer_t *linearLayerInitOwning(linearInit_t *init, layerQuant_t *lq) {	
/* same setup as Borrowing */	
cfg->forwardQ = deepCopyQuantization(lq->forwardMath);	<-- deep copy
cfg->weightGradQ = deepCopyQuantization(lq->backwardMath);	<-- deep copy
cfg->biasGradQ = deepCopyQuantization(lq->backwardMath);	<-- deep copy
cfg->propLossQ = deepCopyQuantization(lq->backwardMath);	<-- deep copy

<code>cfg->ownsQuantizations = true;</code>	
<code>return layer;</code>	
<code>}</code>	

Four separate deepCopys even if backwardMath is same pointer in all four. Simplifies freeLinearLayer: no pointer dedup needed at teardown.

Chapter 12 -- freeLinearLayer

void freeLinearLayer(layer_t *linearLayer) {	
if (linearLayer == NULL) { return; }	<-- null guard
linearConfig_t *cfg = linearLayer->config->linear;	
if (cfg->weights != NULL) {	<-- free param+grad
freeParameter(cfg->weights); }	
if (cfg->bias != NULL) { freeParameter(cfg->bias); }	<-- free param+grad
if (cfg->ownsQuantizations) {	<-- only for Owning variant
if (cfg->forwardQ != NULL) {	
freeReservedMemory(cfg->forwardQ->qConfig);	<-- free inner first
freeReservedMemory(cfg->forwardQ);	<-- then outer
}	
if (cfg->weightGradQ!=NULL &&	<-- dedup guard
cfg->weightGradQ!=cfg->forwardQ) {	
freeReservedMemory(cfg->weightGradQ->qConfig);	
freeReservedMemory(cfg->weightGradQ);	
}	
/* same for biasGradQ and propLossQ */	
}	
freeReservedMemory(cfg);	<-- free config struct
freeReservedMemory(linearLayer->config);	<-- free layerConfig
freeReservedMemory(linearLayer);	<-- free layer struct
}	

CONCEPT: Teardown Order: Inner Before Outer

1. Free grad tensor data, shape, struct
2. Free param tensor data, shape, struct
3. Free parameter_t struct
4. Free linearConfig_t
5. Free layerConfig_t
6. Free layer_t

General rule: always free children before parents.

Never access a struct field after freeing the struct.

CONCEPT: Defensive Dedup in Free

Owning factory allocates four SEPARATE copies, so no dedup needed in practice.

Guards (`weightGradQ != forwardQ`, etc.) are defensive for future maintenance.

Two-step free per quantization:

1. Free inner `qConfig` (pointed-to sub-struct)
2. Free outer `quantization_t` (wrapper struct)

Always free the pointed-to object before the pointer-holding object.

Chapter 13 -- RigL Integration Architecture

The codebase is designed to support RigL. Hook points exist; full wiring is planned.

CONCEPT: RigL Algorithm (Evci et al. 2020)

Maintains adaptive sparse weight mask during training.

At each RigL update step (every T iterations):

PRUNE: zero out k% of ACTIVE weights with smallest |magnitude| -> mask=0

GROW: activate k% of INACTIVE positions with largest |gradient| -> mask=1, weight=0

Between steps: only active weights (mask=1) participate in forward/backward.

Fixed sparsity ratio. Adaptive selection of WHICH weights are sparse.

Key insight: gradient magnitude at inactive positions predicts importance.

Growing high-gradient inactive weights is principled, not random.

```
/* sparsity_t hook in tensor_t: */
typedef struct sparsityConfig { /* empty placeholder */ } sparsityConfig_t;
typedef struct sparsity { sparsityConfig_t *sparsityConfig; } sparsity_t;

typedef struct tensor {
    uint8_t *data;
    shape_t *shape;
    quantization_t *quantization;
    sparsity_t *sparsity; /* <-- RigL hook */
} tensor_t;
```

```
/* Future linearConfig_t with RigL mask: */
typedef struct linearConfig {
    parameter_t *weights;
    parameter_t *bias;
    tensor_t *mask; /* BOOL tensor [outF,inF] */
    /* 1 bit per weight: 1=active, 0=pruned */
    /* 64x512 weights -> 32768 bits -> 4096 bytes mask */
    quantization_t *forwardQ;
    /* ... rest unchanged ... */
} linearConfig_t;
```

CONCEPT: BOOL Tensor for the RigL Mask

tensorBoolGet/Set: 1 bit per weight using bit manipulation.

[64,512] weight tensor: 32768 weights -> 32768 bits -> 4096 bytes.

Compare: 32768 x 4 bytes = 128 KB for float weights. Mask is 32x smaller.

Bit layout: element i at byte $i//8$, bit $i\%8$.

getBitmask, readByte, writeByte, byteConversion in Tensor.c support this.

ARM Cortex-M7: bitwise AND/OR with precomputed mask = 1-2 clock cycles.

```
/* Future masked forward (pseudocode): */
void linearForwardSparseFloat(tensor_t *w, tensor_t *mask,
    tensor_t *b, tensor_t *input, tensor_t *output) {
    size_t outF=w->shape->dimensions[0]; size_t inF=w->shape->dimensions[1];
    float *W=(float*)w->data; float *I=(float*)input->data; float *O=(float*)output->data;
    for (size_t o=0; o<outF; o++) {
        float sum=0.0f;
        for (size_t i=0; i<inF; i++) {
            if (tensorBoolGet(mask, o*inF+i)) { /* mask=1: active */
                sum += W[o*inF+i] * I[i];
            } /* mask=0: pruned, contributes 0 */
        }
        O[o] = sum + ((float*)b->data)[o];
    }
}
```

```
/* RigL update step (pseudocode): */
void rigLUpdateStep(tensor_t *weights, tensor_t *grads,
    tensor_t *mask, float sparsityRatio) {
    size_t N = calcNumberOfElementsByTensor(weights);
    size_t k = (size_t)(N * sparsityRatio * DROP_FRACTION);

    /* PRUNE: find k active weights with smallest |w| */
    /* collect (|w[i]|, i) for all i where mask[i]=1 */
    /* sort ascending by |w|, take bottom k */
    /* set mask[i]=0 for those k positions */

    /* GROW: find k inactive positions with largest |grad| */
    /* collect (|grad[i]|, i) for all i where mask[i]=0 */
    /* sort descending, take top k */
    /* set mask[i]=1, weights[i]=0 */
}
```

CONCEPT: Gradient Flow Through Sparse Layers

During backward with mask:

$dW[i,j] = \text{loss}[j] * \text{input}[i]$ if $\text{mask}[i,j]=1$ (active)

$dW[i,j] = 0$ if $\text{mask}[i,j]=0$ (but still computed and stored!)

Key: compute gradients for ALL positions including inactive ones.

The inactive gradient is used by GROW step to find high-gradient positions.

Current backward (backwardFloat/SymInt32) accumulates into weightGrads regardless of mask -- already gradient-complete for RigL integration.

Chapter 14 -- End-to-End Training Walkthrough

2-layer network: batch=2, inF=4, hidF=3, outF=2, FLOAT32.

```
/* Setup */
linearInit_t init1={.inFeatures=4,.outFeatures=3,.bias=BIAS_DEFAULT};
linearInit_t init2={.inFeatures=3,.outFeatures=2,.bias=BIAS_DEFAULT};
layerQuant_t lq={.forwardMath=&floatQ,.backwardMath=&floatQ,
.weightStorage=&floatQ,.biasStorage=&floatQ};

layer_t *layer1=linearLayerInit(&init1,&lq); /* [4->3] Kaiming init */
layer_t *layer2=linearLayerInit(&init2,&lq); /* [3->2] Kaiming init */

/* Forward */
linearForward(layer1,&input,&hidden); /* hidden=input*W1^T+b1 [2,3] */
linearForward(layer2,&hidden,&output); /* output=hidden*W2^T+b2 [2,2] */

/* Backward (dOutput computed by loss function) */
linearBackward(layer2,&hidden,&dOutput,&propLoss2);
/* weightGrad2+=dOutput^T*hidden biasGrad2+=sum(dOutput) propLoss2=dOutput*W2 */
linearBackward(layer1,&input,&propLoss2,&propLoss1);
/* weightGrad1+=propLoss2^T*input biasGrad1+=sum(propLoss2) propLoss1=propLoss2*W1 */
/* SGD: W -= lr*weightGrad; b -= lr*biasGrad */
```

Chapter 15 -- Glossary

Arena allocator	Bump-pointer allocator using static buffer. O(1) alloc. No fragmentation. Zero-initialized. reserveMemory() in ODT.
backwardFloat	Runs all three backward sub-operations (weight grad, bias grad, prop loss) for FLOAT32 case.
Borrowing	Factory stores raw pointers without owning them. ownsQuantizations=false. Caller keeps objects alive.
deepCopyQuantization	Allocates new quantization_t + qConfig, copies all fields. Used in linearLayerInitOwning.
FQT	Fixed-point Quantization Training. Trains with integer arithmetic + scale factors. SYM_INT32 is the ODT FQT type.
getQLike	Clones quantization_t (type+qConfig) into fresh arena allocation. Tensor takes ownership.
gradInit	Allocates a gradient tensor with same shape as param tensor and specified quantization type.
Issue #187	propLoss type guard: matmulFloat32 writes float bits; SYM_INT32 propLoss has NULL qConfig -> segfault. Guard exits early.
Kaiming Uniform	He et al. 2015 weight init. $W \sim \text{Uniform}[-\text{gain}/\sqrt{\text{fanIn}}, \text{gain}/\sqrt{\text{fanIn}}]$. Prevents vanishing gradients.
orderOfDimensions	Array in shape_t. Maps logical dim d to physical dim. Identity=[0,1]=normal. Swap=transposed.
Owning	Factory deep-copies all quantization_t. Caller can free originals. ownsQuantizations=true.
parameter_t	Bundles param tensor + grad tensor. parameterInit(param,grad). Both always same shape.
propLoss	Gradient flowing to previous layer. For linear: propLoss=loss*W. Shape [batch,inF].
qMaxBits	Max bit width for SYM_INT32 mantissas during training. qMaxBits=16 -> int16 range -> int32 accumulation safe.
reserveMemory	ODT arena allocator. Returns next available bytes. Zero-initialized. O(1).
RigL	Evci et al. 2020. Adaptive sparse training: prune small w , grow high grad positions every T steps.
sparsity_t	Placeholder hook in tensor_t for RigL mask config. Currently empty struct.
SR_HALF_AWAY	Stochastic Rounding. Adds uniform noise before rounding. Unbiased quantization -> FQT convergence.

SYM_INT32	real=mantissa*scale. mantissa: int32_t. scale: float in symInt32QConfig_t. FQT quantization type.
transposeTensor(w,0,1)	Swaps orderOfDimensions[0] and [1]. Zero-copy. No data moves.
VLA	Variable Length Array (C99). int arr[n] with runtime n. Allocated on STACK. Risk of overflow on MCU.
Zero-copy transpose	transposeTensor only changes 2 integers in shape metadata. Physical data untouched.